Working SoC RTL prototype

A reproducible integration demo with seven passing, ordinary self-checking testbenches. The system is working in RTL simulation; it is not yet validated on an FPGA.

What is integrated

- **CPU and bus:** a five-stage RV32I core has separate instruction and data AHB masters connected through the Roa Logic interconnect. A new AHB-to-APB bridge exposes memory-mapped peripherals.
- **Memory:** 256 KiB instruction/boot SRAM at 0x8000_0000 and 64 KiB data SRAM at 0x4000_0000. Firmware currently starts from a hex image in SRAM.
- **Mission subsystem:** eight event regions, priority selection, observation scheduling, a 256-entry FIFO, mode control and CRC-protected telemetry.
- **Peripherals:** GPIO, UART 8N1, single-line SPI mode 0, two imported PULP APB timers, a CLINT and a single-target PLIC.
- **Interrupt-capable CPU variant:** machine external, timer and software interrupt inputs connect to CSRs and a tested trap/return path. Original supplied core files are retained separately.

Memory-mapped regions

Base	Block	Key use
0x8000_0000	Boot SRAM - 256 KiB	Instruction fetch and data access
0x4000_0000	Data SRAM - 64 KiB	Observation firmware data
0x5000_0000	Mission AHB - 4 KiB	Configure, trigger and monitor
0x6000_0000	GPIO APB - 4 KiB	Input, output and pin interrupt
0x6000_1000	UART APB - 4 KiB	Serial telemetry output
0x6000_2000	SPI APB - 4 KiB	Mode-0 telemetry output
0x6000_3000	Timer APB - 4 KiB	Two timers; compare IRQ
0x6000_4000	CLINT APB - 4 KiB	MSIP and MTIP
0x6000_5000	PLIC APB - 4 KiB	Source priority, claim/complete

Access rules: instruction fetch is restricted to boot SRAM; the data master reaches SRAM and MMIO. Mission/APB registers use aligned 32-bit accesses. SRAM supports byte, halfword and word accesses.

Mission behavior and interface

One event becomes one tagged sensor request, one stored observation and one telemetry packet.

Six connected mission functions

- **Event detector:** synchronizes eight asynchronous inputs, detects rising edges, and records sticky pending events. Software can also inject region events.
- **Region priority file:** holds ID, priority and enable for each region. Higher numeric priority wins; the lower region index breaks equal-priority ties.
- **Mode manager:** chooses nominal, priority, emergency or communication behavior. Region 7 is the default emergency event source.
- **Observation scheduler:** claims a pending region, sends a request with a 16-bit tag, waits for a matching 32-bit sample, and times out if no valid response arrives.
- **Data storage:** a volatile FIFO stores region ID, priority, timestamp and sample. Full storage blocks new claims. Retention blocks routine flush; force flush clears it.
- **Telemetry:** emits an 18-byte packet with CRC-16/CCITT-FALSE. Ready/valid backpressure preserves each byte until UART or SPI accepts it.

Sensor and packet boundary

- **Sensor request:** valid/ready, region ID, priority, event time and request tag. The response returns valid, data and the same tag. The testbench supplies the sensor value; acquisition hardware is still needed.
- **Packet:** sync EB 90, version 01, payload length 0A, sequence, region, priority, 32-bit time, 32-bit data and a 16-bit CRC over bytes 2 through 15. Multi-byte fields are sent most significant byte first.

Software control - selected mission registers

Offset	Register	Function
0x00	CONTROL	Run, storage, retention, communication and UART/SPI select
0x04	TRIGGER	Write a bit to inject a region event
0x0C	TIMEOUT	Observation wait limit in cycles
0x10 / 0x18	INDEX / CFG	Select region; write ID, priority and enable
0x20 / 0x24	PENDING / CLEAR	Read event and region pending; clear bits
0x28 / 0x30	COMMAND / STATUS	Flush/acknowledge; read FIFO count and flags

Verification scope: the test suite directly checks two-region priority selection, stored observations, packet payload and CRC. Emergency mode, all timeout cases and saturation corner cases need broader coverage.

Verification evidence

Every result below comes from a conventional self-checking RTL testbench. No UVM testbench is used.

Test	Checked behavior
tb_mission	Priority dispatch, two stored samples, two packets, payload and CRC
tb_soc_demo	CPU SRAM and GPIO readback, imported timer IRQ, mission MMIO and all UART pin bytes
tb_soc_spi	Firmware mission flow to full SPI mode-0 MOSI packet
tb_interrupt_blocks	APB PLIC priority, claim/complete/rearm and CLINT MSIP/MTIP
tb_soc_irq	Software interrupt enters CPU handler with mcause; mret returns
tb_soc_plic	GPIO -> PLIC -> machine external trap -> claim/complete -> mret
tb_soc_timer_irq	CLINT timer compare -> machine timer trap -> handler -> mret

A five-step demo narrative

- **1.** Firmware starts in boot SRAM and writes/reads both GPIO and data SRAM through AHB/APB.
- **2.** Firmware programs a mission region and injects a software event.
- **3.** The scheduler requests a tagged sensor sample; the testbench responds with 0xDEADBEEF.
- **4.** The observation enters storage and becomes an 18-byte CRC-protected UART packet. A second run verifies SPI MOSI.
- **5.** Independent firmware tests show software, timer and PLIC interrupts entering handlers and returning with mret.

Reproduce the result

From the repository root, with Python 3, Icarus Verilog, Verilator and a C++ compiler:

```
bash scripts/run_tests.sh
```

The script regenerates firmware images, runs all seven tests and writes compile/simulation logs to build/. It reports: **ALL 7 NON-UVM TESTS PASSED**.

Status, gaps and next steps

The code is a working simulation prototype. The items below define what remains before hardware or broad compliance claims.

Current limits

- **Sensor hardware:** the request/response contract is RTL, but a physical sensor acquisition block and application firmware are not included.
- **Boot and debug:** firmware is preloaded in SRAM. Physical SPI flash boot and JTAG debug are not integrated.
- **Board interfaces:** UART-USB hardware, IO pad ring, switch/LED wiring, clocks/reset plan, pin assignments and FPGA constraints remain.
- **SPI capability:** the integrated SPI block is single-line mode 0. Full QSPI/flash modes are future work. The imported SPI repository copy lists source dependencies that are absent locally.
- **Verification depth:** seven directed smoke/regression tests pass; protocol compliance, full CPU ISA testing, stress, coverage and UVM remain future verification tasks.

Practical next sequence

- **1. Firmware:** define a repeatable sensor sampling scenario and add register-level driver code.
- **2. Boot path:** choose flash part/model and verify reset-to-firmware execution from the intended source.
- **3. FPGA:** select a board, map pins/clock/reset, run Vivado synthesis and timing, then test physical UART/SPI/GPIO.
- **4. Verification:** expand normal tests for backpressure, timeout, interrupts and concurrency; then build UVM agents and coverage.

Team access and source provenance

github.com/Shivay315/1TOPS-SoC-RTL (private repository; teammates require collaborator access).

Supplied RTL remains in rtl/core, rtl/bus, rtl/top/soc_top.sv and rtl/mission. Imported IP is under third_party. New integration RTL is separated into rtl/core_irq, rtl/mission_v1, rtl/periph and rtl/top/emu_*.

Meeting statement: "We have a working RISC-V SoC RTL prototype verified in simulation. The CPU, buses, SRAM, mission observation and telemetry path, peripherals and machine interrupts pass seven non-UVM tests. Physical sensor, flash boot, debug and FPGA validation are the next steps."